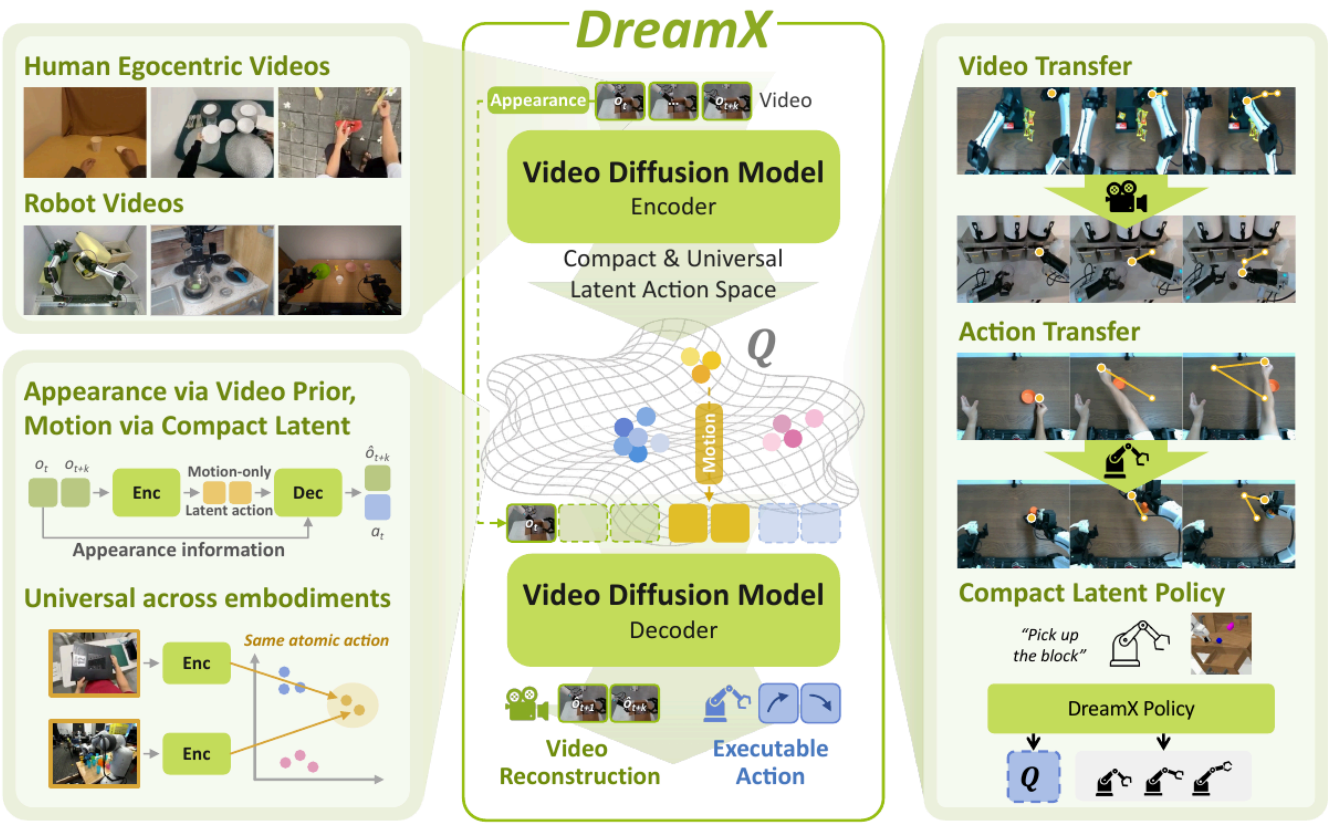
DreamX: 분석 리포트

게시용 버전 · 원문 https://arxiv.org/abs/2611.05377 · arXiv:2611.05377v1 · KAIST, NVIDIA (Jisoo Kim, Chaehyun Kim, ⋯ Fu-En Yang, Seungryong Kim) · 공식 코드 미공개(cvlab-kaist/DreamX @ 056bd46, README만) · 분석일 2026-11-09 · 논문 그림 © 저자, CC BY 4.0



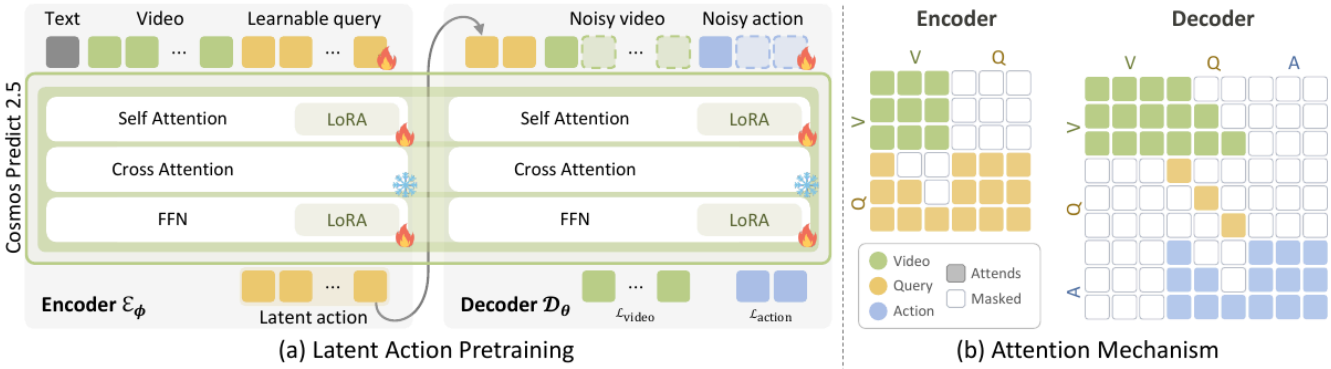
논문 Fig. 1: 사람 1인칭 영상과 로봇 영상에서 영상 diffusion 모델로 공통 latent action을 학습하고, 영상 전이·행동 전이·압축 latent 정책에 쓴다.

핵심 요약

1. **무엇을 했나:** Cosmos-Predict2.5 2B에 learnable query를 붙여 latent frame당 4개의 latent-action 토큰을 읽고(Eq. 2), 같은 백본을 decoder로 써서 첫 프레임과 이 토큰만으로 미래 영상을 재구성한다(Eq. 3-4). 로봇 클립에는 행동 보조 손실을, 사람 클립(21.4%)에는 재구성만 쓴다. 2.19k시간, 78k step.
2. **표 수치는 맞다:** Table 1 평균(DiLA 회귀 0.3875 → 0.38만 경계), Table 3 SR 96.7/50.0, Table 7 Avg. Len., 데이터 합계 2,188 h가 재현된다. LARYBench 회귀 오차는 기존 최고 대비 44-65% 낮고, 32차원 압축 코드(FAE-32)도 모든 기준선보다 낫다.
3. **정렬의 근거는 "같은 시점"에 몰려 있다:** 재구성 손실은 사람·로봇 latent가 같은 좌표에 놓이기를 요구하지 않는다. 선형 toy에서 공유 encoder는 이미지 공간 움직임이 일치할 때만 정렬되고(전이 오차 = $2(1-\cos\theta)$), 짝 클립 5%면 θ와 무관하게 정렬된다. Fig. 5·6의 사람 영상은 로봇과 같은 위쪽 시점이고, 1인칭 → 3인칭 전이는 평가되지 않았다.
4. **탐침 점수는 정렬을 재지 못한다:** toy의 모든 조건에서 embodiment별 탐침 R^2는 1이다. LARYBench의 높은 점수는 정보량의 근거이지 공통 좌표의 근거가 아니다. 영상 모델만 중간 프레임을 받았고, 학습 데이터의 37.4%(AgiBot)가 평가셋과 같은 출처다.
5. **통제되지 않은 경로:** encoder는 지시문을 받고 decoder는 받지 않는다. 빈 지시 ablation이 없어 전이의 어디까지가 움직임이고 어디까지가 과제 의미인지 가를 수 없다. query를 0으로 바꿔도 움직임이 46% 남고, 채널 표준편차만큼의 잡음에도 94.6%가 유지된다(Table 8).
6. **"300×"는 토큰 개수다:** 토큰당 2,048차원이라 프레임당 8,192개 실수, VAE latent 대비 9.4×다. 실물 정책 우위는 push 28/30 대 2/30($p = 3\times10^{-12}$)으로 확실하지만 reach는 차이가 없고($p = 0.49$), "사람 영상만으로 100%"는 로봇 시연 8/10

과 구분되지 않는다(p = 0.47).

7. **구현 가능성:** 코드·가중치가 없어 공식 재현은 불가능하다. 이 맥(M4 Pro 24 GiB, MPS)은 표 재계산과 toy까지 했고, 공개 백본 단독 추론(4.6 GB)은 설치·다운로드 승인 후 조건부(미검증, 공식 스택은 CUDA·Linux 전제), 학습은 불가능하다. 재구현은 query + LoRA 구조라 중간 난이도이지만 행동 branch와 행동 공간 통일 방식이 빠져 있다.



논문 Fig. 2: (a) query가 백본의 시공간 표현을 읽어 latent action을 만들고, 같은 백본의 decoder가 첫 프레임과 latent action으로 미래 영상(과 레이블이 있으면 행동)을 재구성한다. (b) encoder·decoder attention 마스크.

01. 논문 분석: DreamX (arXiv:2611.05377)

- **제목:** *DreamX: Video Diffusion Model is a Universal Latent Action Learner*
- **저자:** Jisoo Kim, Chaehyun Kim, Junsu Kim, Jiho Park, Seongchan Kim, Eunbeen Hong, Soowon Son, Fu-En Yang, Seungryong Kim (KAIST, Fu-En Yang은 NVIDIA)
- **공개:** arXiv v1(2026-11-03). 본문 10쪽, 부록 A-D와 참고문헌을 합쳐 25쪽입니다. 프로젝트 페이지는 https://cvlab-kaist.github.io/DreamX/ 입니다.
- **코드·가중치:** 공개되지 않았습니다. 프로젝트 페이지에는 "Code soon"만 있고, `cvlab-kaist/DreamX` 저장소(@ `056bd46`)에는 README와 티저 그림뿐입니다(2026-11-09 확인).
- **라이선스:** arXiv abs 페이지 기준 **CC BY 4.0**입니다. PDF 안에 이와 충돌하는 권리 표기(© 문구 등)는 없습니다. 그래서 출처를 밝히고 게시본에도 논문 그림을 넣었습니다.

한 줄 요약: 사전학습된 영상 diffusion 모델(Cosmos-Predict2.5 2B)에 학습 가능한 query 토큰을 붙여 영상 특징에서 **latent frame당 4개의 latent-action 토큰**을 읽어 내고, 같은 백본을 decoder로 써서 첫 프레임과 이 토큰만으로 미래 영상을 재구성하도록 학습합니다. 이렇게 얻은 latent action을 사람·로봇 사이의 공통 행동 인터페이스로 쓰고, 영상 전이, 행동 전이, 정책 학습에 활용합니다.

1. Abstract와 문제 설정

문제. 사람 영상에는 로봇 행동 레이블이 없고, 로봇끼리도 제어 공간이 다릅니다. 그래서 같은 행동이 embodiment마다 다른 저수준 행동으로 기록됩니다(§1). latent action model(LAM)은 연속 프레임의 변화를 압축한 latent action으로 이 문제를 풀려 하지만, 재구성 정확도만으로는 latent action이 "행동"만 담는다는 보장이 없습니다. 재구성에 도움이 되면 시점이나 장면 외관 같은 도메인 정보도 함께 담깁니다(§1, Nikulin et al. 2025 인용).

주장.

1. 영상 diffusion 모델의 시공간 특징은 움직임·대응·기하를 이미 담고 있으므로, 여기서 latent action을 직접 읽으면 좋습니다(§1, §2).
2. 같은 모델의 생성 prior가 외관을 책임지므로, decoder가 첫 프레임과 latent action으로 미래를 재구성하게 하면 latent action은 움직임에 집중하게 됩니다(§3.2).
3. 그 결과 LARYBench에서 LAM 중 최고 성능을 냈고(Table 1), 짝지은 학습 데이터 없이도 임의 embodiment 간 영상 전이와 사람→로봇 zero-shot 행동 전이가 됩니다(§4.2-4.3).
4. 압축 코드를 예측 대상으로 하는 정책은 미래 영상을 생성하는 정책(Mimic-Video)보다 성공률이 높고 19.1배 빠릅니다(Table 3).

기여(§1). (i) 영상 diffusion 모델의 시공간 prior와 생성 prior로 범용 latent action을 학습하는 틀, (ii) 짝지은 데이터 없는 영상·행동 전이와 압축 latent 코드를 예측 대상으로 하는 정책.

2. Method

2.1 백본: Cosmos-Predict2.5 (§3.1, Eq. 1)

causal 영상 autoencoder가 클립을 latent frame N개 $V = (V_0, \cdots, V_{N-1})$로 압축합니다. V_0는 첫 프레임만 담습니다. 이 latent를 공간 패치 토큰으로 나눠 시공간 attention으로 모델링하고, 첫 프레임과 텍스트 등의 조건 c 아래에서 flow matching으로 학습합니다. $x_\tau = (1-\tau)x + \tau\varepsilon$, 손실은 $\| v_\theta(x_\tau, \tau; c) - (\varepsilon - x) \|^2$입니다(Eq. 1).

2.2 Encoder: query 토큰으로 latent action 읽기 (§3.2, Eq. 2, Fig. 2)

- 학습 가능한 query 토큰 q를 영상 토큰 열에 이어 붙여, 백본의 시공간 attention을 함께 통과시킵니다.
- latent frame V_g마다 query K = 4개(q_g)를 배정하고, 문맥화된 출력이 latent-action 그룹 Q_g가 됩니다. 모든 그룹을 한 번의 forward로 뽑습니다: **Q = E_φ(q; V, ℓ)** (Eq. 2).
- **ℓ은 언어 지시이고, 백본의 원래 cross-attention으로 encoder에 들어갑니다.** decoder에는 텍스트가 들어가지 않습니다. 이 차이는 §4에서 다시 다룹니다.
- attention 마스크(Fig. 2b): query → 영상은 프레임 causal이라 Q_g는 V_{0:g}만 봅니다. query끼리는 제한이 없습니다.
- encoder는 사전학습 백본에 rank-16 LoRA를 붙여 학습합니다(백본 가중치는 고정, 부록 A.2).

2.3 Decoder: 생성 재구성과 보조 행동 손실 (§3.2, Eq. 3-4)

- 같은 백본을 별도 rank-16 LoRA로 decoder로 씁니다. 조건은 V_0와 Q_{0:N−1}이고 텍스트는 넣지 않습니다.
- 행동 레이블이 있는 클립에서는 고유수용 상태 p_0를 받아 행동 chunk도 함께 decode합니다: $\hat{V}, \hat{A}$ = D_θ(Q_{0:N−1}, V_0, p_0) (Eq. 3).
- decoder 마스크: query는 조건으로만 쓰이고 아무것도 보지 않습니다. 영상 프레임 V_t는 Q_{0:t}를, 행동 chunk A_m은 Q_{0:m}을 보며, 영상과 행동 스트림은 서로를 보지 않습니다.
- 손실: L = L_video + λ L_action, 행동 레이블이 없는 클립은 λ = 0입니다(Eq. 4). 레이블이 있으면 두 손실의 가중치가 같습니다(부록 A.2).

2.4 전이 (§3.3, Eq. 5, Fig. 3a)

원본 영상에서 고정된 encoder로 Q_src = E_φ(q; V_src, ℓ_src)를 뽑고, 두 가지로 씁니다.

- **영상 전이:** $\hat{V}$_tgt = D_θ(Q_src, V_0^tgt). 대상 장면의 첫 프레임에 원본 움직임을 입힙니다.
- **행동 전이:** Â_tgt = D_θ(Q_src, p_0^tgt). 대상 로봇의 초기 상태에서 실행할 행동을 decode합니다.

논문은 이 전이 능력이 학습 목표에 없던 **창발 성질**이라고 설명합니다(§3.3). 추가 학습은 하지 않습니다.

2.5 압축과 정책 (§3.4, Eq. 6-9, 부록 A.3-A.4)

- **압축(FAE):** latent-action 토큰은 개수는 적지만 각 토큰이 백본 폭(2,048차원)을 그대로 가집니다. 논문도 이 때문에 직접 diffusion하기에는 차원이 높다고 적습니다(§3.4). 그래서 encoder를 고정하고 변분 bottleneck으로 토큰마다 2,048 → 32 차원으로 줄입니다. 손실은 $\|\hat{Q} - Q\|^2 + \beta$ KL이고 β = 0.01입니다(Eq. 8-9). encoder는 self-attention 블록 하나, decoder는 6층 transformer, 학습은 50k step입니다. 정책 학습에는 사후 평균 Z = μ_ψ(Q)를 씁니다.
- **정책:** π(Z, A | V_0, ℓ, p_0)(Eq. 6). 같은 영상 백본이 현재 프레임과 지시에서 압축 코드 Z를 생성하고, 별도의 Action DiT가 생성기의 latent-action 특징, 관찰, 지시, 상태를 보고 행동을 생성합니다. 행동 토큰은 latent 특징을 보지만 반대는 아닙니다. 손실은 L_latent + λ L_action이고, 사람 영상은 λ = 0입니다(Eq. 7).
- **생성기 사전학습:** DreamX 사전학습 데이터의 약 1/3로 rank-128 LoRA를 100k step 학습합니다(H200 2장, 배치 60). Action DiT는 따로 사전학습하지 않고 과제별 학습 때 함께 학습합니다(부록 A.4).

2.6 학습 설정 (§4 구현 세부, 부록 A.1-A.2)

항목	값
백본	Cosmos-Predict2.5 2B, 480 × 640
클립	5-29프레임(4 간격) = latent frame 2-8개, query 최대 32개
학습	encoder·decoder 각각 rank-16 LoRA, 78,000 step, LoRA·query 학습률 1e-4, 나머지 5e-5, warm-up 500

항목	값
데이터	2.19k시간, 6개 데이터셋(Fig. 9): AgiBot 818.2 h, MolmoAct2-BimanualYAM 502.3 h, DROID 410.1 h, EgoDex 350.5 h, EgoVerse 59.0 h, Bridge 47.9 h. 사람 영상 21.4%
미보고	GPU 종류·수와 학습 시간(사전학습), 배치 크기, seed, 행동 레이블이 있는 클립의 비율

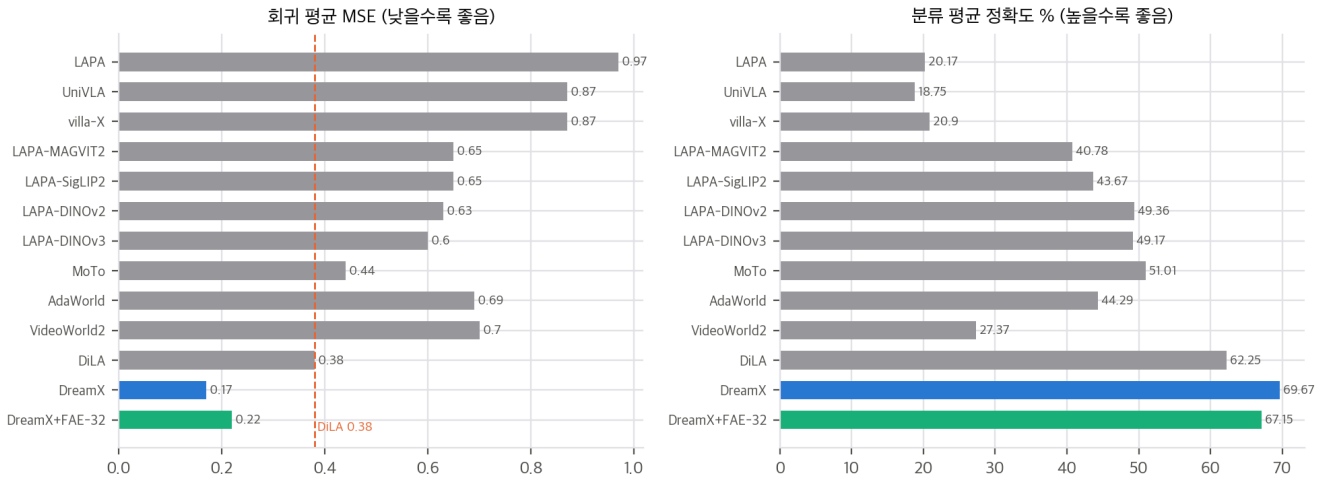
3. Experiments

3.1 표현 품질: LARYBench (§4.1, Table 1, 부록 B.4)

모든 표현 모델을 고정하고 공통 attentive probe를 학습해, 말단 궤적 회귀(정규화 MSE)와 행동 분류(정확도)를 평가합니다.

방법	회귀 평균 ↓	분류 평균 ↑
LAPA / UniVLA / villa-X (프레임 2장)	0.97 / 0.87 / 0.87	20.17 / 18.75 / 20.90
LAPA-DINOv3 (프레임 2장)	0.60	49.17
MoTo / AdaWorld / VideoWorld2 (영상)	0.44 / 0.69 / 0.70	51.01 / 44.29 / 27.37
DiLA (영상, 기존 최고)	0.38	62.25
DreamX	**0.17**	**69.67**
DreamX + FAE-32	0.22	67.15

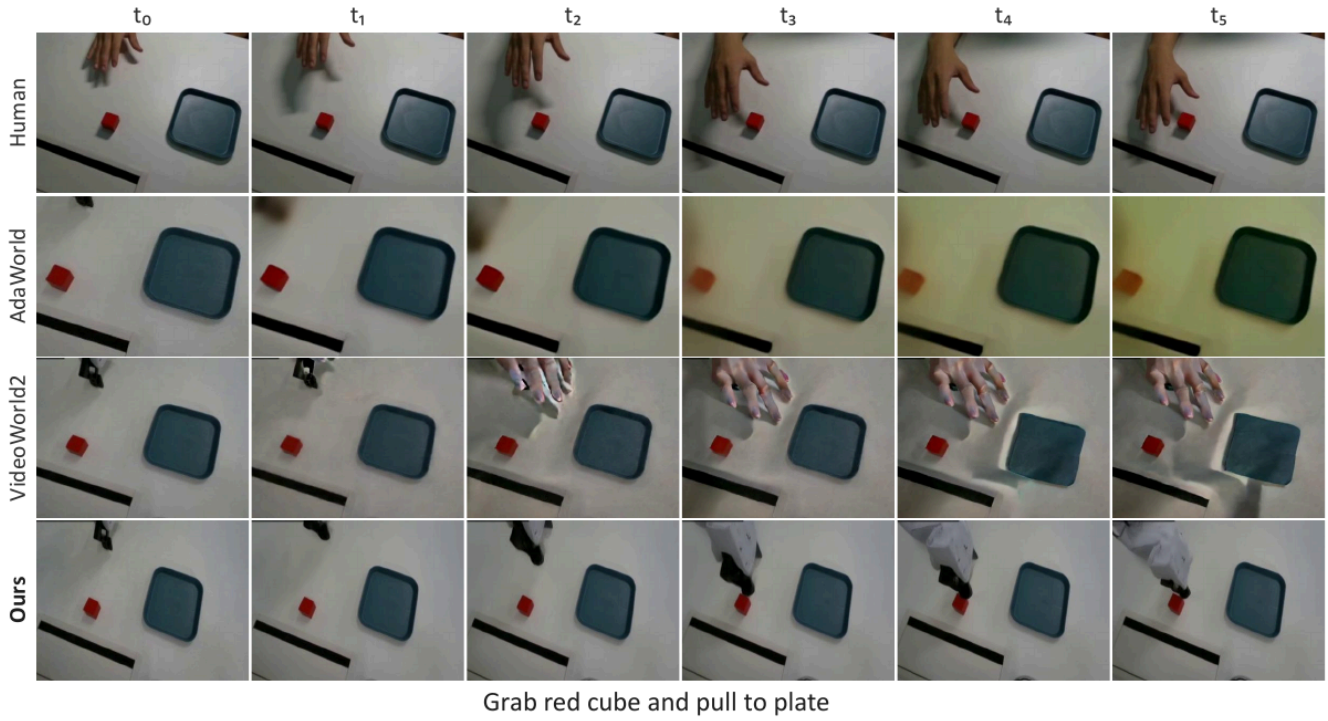
- DreamX는 8개 열 모두에서 1위이고, FAE-32 압축 코드도 모든 기준선보다 낫습니다.
- **비교 조건이 같지 않습니다.** 회귀 벤치마크는 원래 표본당 프레임 2장만 주는데, 저자들이 메타데이터로 원본 영상의 중간 프레임까지 복원해 영상 기반 모델에만 넣었습니다(부록 B.4). 표의 위쪽 그룹(프레임 2장)과 아래쪽 그룹(영상)은 입력 정보량부터 다릅니다. 논문도 표를 두 그룹으로 나눠 두었습니다.
- encoder가 언어 지시 ℓ을 받는데(Eq. 2), LARYBench 특징을 뽑을 때 지시를 넣었는지, 넣었다면 무엇을 넣었는지 적혀 있지 않습니다. 분류 과제라면 지시문이 정답 정보를 줄 수 있습니다(`04` C9).
- 평균을 다시 계산하면 DiLA 회귀 평균만 0.3875로 표의 0.38과 경계에 있습니다(나머지는 반올림 안에서 일치, `04` C1).



출처: Table 1 (LARYBench). 회색 = 기준선, 파랑 = DreamX, 초록 = FAE-32 압축 코드(토큰당 32차원). 탐침은 벤치마크마다 따로 학습.

Table 1 다시 그림: 회귀 평균 MSE와 분류 평균 정확도. 파랑 DreamX, 초록 FAE-32 압축 코드, 회색 기준선.

3.2 영상 전이 (§4.2, Table 2, Fig. 4-5, 부록 B.5)



논문 Fig. 5: HumanAndRobot 벤치마크의 사람→로봇 zero-shot 영상 전이. 사람과 로봇이 같은 위쪽 시점에서 찍혀 있습니다.

- **임의 embodiment 간 전이(Fig. 4):** 같은 로봇의 다른 환경, 다른 로봇, 사람→로봇, DAVIS 일반 영상까지 정성 예시만 있습니다.
- **사람→로봇 zero-shot(Table 2):** HumanAndRobot 21개 과제에서 과제당 48클립, 총 1,008클립을 씁니다. 각 방법은 사람 영상과, 짝지은 로봇 녹화의 첫 프레임을 받습니다. 나머지 로봇 프레임은 평가 기준으로만 씁니다. 첫 20프레임에서 조건 프레임을 뺀 19프레임으로 평가합니다.

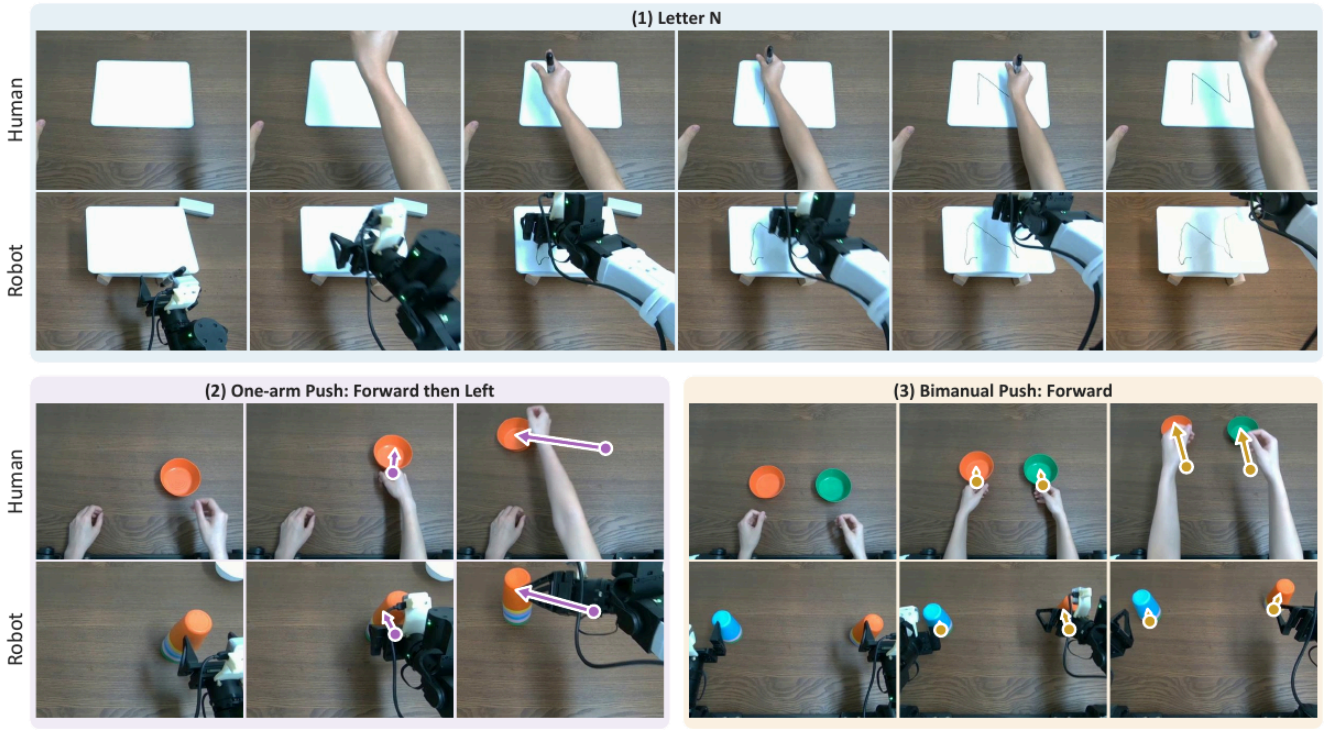
방법	FVD ↓	LPIPS ↓	PSNR ↑	SSIM ↑
AdaWorld	170.19	0.344	22.95	0.846
VideoWorld2	241.92	0.299	22.53	0.881
DreamX	**57.44**	**0.179**	**26.52**	**0.917**

- 기준선은 전이 과제용으로 학습되지 않았고 학습 데이터도 다릅니다. "첫 프레임을 그대로 복사"하거나 query를 0으로 둔 자명한 기준선은 없습니다. PSNR·SSIM은 정지 배경이 대부분을 차지하므로, 이런 기준선이 있어야 움직임 전이 몫을 가를 수 있습니다(`04` C2).

3.3 행동 전이 (§4.3, Fig. 6, 부록 B.2)

사람 시연에서 뽑은 latent action을 로봇 초기 상태와 함께 decode해서 **시각 피드백 없이 open-loop로** 실행합니다. 글자 "N" 쓰기(오른쪽 그리퍼에 마커), 한 팔로 "앞으로 민 뒤 왼쪽으로" 밀기, 양팔로 앞으로 밀기를 보입니다.

- 정량 결과(시행 수, 궤적 오차, 성공률)는 없습니다. 정성 사례만 있습니다.
- Fig. 6의 사람 시연은 로봇 카메라와 **같은 위쪽 시점, 같은 작업대**에서 찍혔습니다. 1인칭 사람 영상에서 3인칭 로봇으로 가는 경우처럼 시점이 다른 전이는 시험하지 않았습니다. 이 차이가 왜 중요한지는 `03` §1의 toy에서 다룹니다.



논문 Fig. 6: 사람 시연에서 뽑은 latent action을 로봇 명령으로 decode해 open-loop 실행. 사람과 로봇이 같은 위쪽 시점에서 찍혔다.

3.4 정책: 실물 서랍 닫기 (§4.4, Table 3, 5, 6, 부록 B.1, B.3, C.1)

- **설정:** 양팔 YAM(팔마다 관절 6개 + 그리퍼 1개, 14차원 절대 명령), 위쪽 시점의 RealSense D405 한 대. 위 서랍만 열림, 아래 서랍만 열림, 둘 다 열림(위 → 아래 순서) 세 설정입니다. 학습 궤적은 45개이고, 설정당 10회 평가합니다. 두 방법 모두 30,000 step 과제별 학습을 했습니다. Mimic-Video는 MolmoAct로 사전학습한 영상 백본에서, DreamX는 사전학습한 latent-action 생성기에서 출발합니다.

방법	위 R/P	아래 R/P	둘 다 R/P	SR(Table 3)	지연(ms)
Mimic-Video	80 / 20	100 / 0	100 / 0	50.0	28,226
DreamX	100 / 80	100 / 100	100 / 100	**96.7**	**1,474**

- SR은 설정마다 reach와 push 완료율을 평균하고 세 설정을 같은 가중치로 평균한 값입니다. 다시 계산하면 정확히 맞습니다(`04` C3).
- **차이는 전부 push에서 나옵니다.** Mimic-Video는 30회 중 28회 서랍에 닿지만 2회만 닫습니다. reach는 30/30 대 28/30으로 차이가 없습니다(Fisher p = 0.49).
- **사람 영상만으로 학습(Table 6):** 위 서랍 설정의 로봇 시연을 사람 영상으로 바꿔도 100/100입니다. 로봇 시연 정책의 위 서랍 push 8/10과 비교하면 p = 0.47로 구분되지 않습니다. 사람 영상의 개수는 보고되지 않았습니다(`04` C4).
- 정책 1회 호출이 1.47초라서 실시간 제어 주기와는 거리가 있습니다.

3.5 Ablation과 분석 (§4.5-4.6, Table 4, 7, 8, Fig. 7-8, 12-13)

- **행동 보조 손실(Table 4):** MolmoAct2·EgoVerse·EgoDex 일부로 학습했을 때 RGB만은 회귀 0.24 / 분류 65.3, RGB + 행동은 0.23 / 64.8입니다. 논문은 "영상만으로 행동 의미를 잡는다"고 해석합니다. 다만 이 표는 LARYBench 탐침 점수만 재고, **행동 손실이 사람→로봇 전이(정렬)에 미치는 영향은 재지 않았습니다**. seed도 1개입니다.
- **CALVIN(Table 7):** 정책의 action head가 latent-action 특징을 못 보게 막으면 Avg. Len.이 2.643 → 2.378로 줄어듭니다. 평가 시퀀스 수는 적혀 있지 않습니다. 절대 수준도 낮습니다(`04` C5).

- **방향 구조(Fig. 12):** RoboCOIN 로봇 10종, 2,362클립의 말단 이동 방향 6가지에 대해, **로봇별 평균을 뺀 방향별 중심점 60개**를 t-SNE로 그렸습니다. 그림 안에 "개별 클립 군집이 아니라 집계 시각화"라고 적혀 있습니다. 사람 클립은 없습니다. 로봇별 평균을 빼는 단계가 바로 embodiment마다 다른 오프셋을 지우는 조작이므로, 이 그림은 "로봇 간 정렬"의 강한 근거가 되지 못합니다.
- **교란 민감도(Table 8, Eq. 10):** 고움직임 클립 8개를 골라 decoder seed 2개로 평가했습니다. 채널 표준편차 크기의 잡음(α = 1)을 더해도 움직임 크기 비율은 0.946으로 거의 그대로입니다. query를 0으로 바꿔도 움직임이 45.6% 남습니다. latent action이 움직임을 조종하는 것은 맞지만, 첫 프레임과 생성 prior만으로도 움직임의 절반 가까이가 나온다는 뜻입니다(`04` C8).
- **attention(Fig. 8):** query가 손과 그리퍼 같은 움직이는 영역을 봅니다. 정성 결과입니다.

4. Limitations

논문에는 한계 절이 따로 없습니다. 윤리 절에서 "평가하지 않은 설정에서는 실패할 수 있다"고만 적었습니다. 아래는 이 분석에서 정리한 한계입니다.

1. **정렬이 왜 생기는지 설명이 없습니다.** 재구성 손실은 embodiment마다 다른 latent 좌표계를 허용합니다. 사람 영상에는 행동 손실이 없으므로(λ = 0), 사람과 로봇의 latent가 같은 좌표에 놓일 압력은 (a) 가중치 공유와 (b) 같은 행동이 이미지에서 비슷하게 움직여 보이는 것뿐입니다. toy(`03` §1)에서 공유 encoder는 이미지 공간의 움직임이 일치할 때(θ ≈ 0)만 정렬되고, 시점 차이로 움직임이 돌아 보이면 전이 오차가 2(1 − cos θ)로 커졌습니다.
2. **전이 증거가 쉬운 경우에 몰려 있습니다.** Fig. 5(HumanAndRobot)와 Fig. 6(실물)의 사람 영상은 로봇 카메라와 같은 위쪽 시점에서 찍었습니다. 학습 데이터의 사람 영상(EgoDex, EgoVerse)은 1인칭인데, 1인칭 → 3인칭 전이의 정량 평가는 없습니다.
3. **텍스트 경로가 통제되지 않았습니다.** encoder는 지시 ℓ을 받고 decoder는 받지 않습니다(Eq. 2, §3.2). 전이할 때 Q_src에는 원본 지시(예: "Grab red cube and pull to plate")의 의미가 들어갈 수 있고, decoder는 그 의미를 대상 장면에서 실현할 수 있습니다. 빈 지시로 encode하는 ablation이 없어서, 전이 중 어디까지가 움직임이고 어디까지가 과제 의미인지 가를 수 없습니다.
4. **"300× 압축"은 토큰 개수 기준입니다.** 각 토큰은 2,048차원이라 프레임당 8,192개 실수이고, VAE latent(프레임당 76,800개로 추정) 대비 9.4배입니다. 진짜 작은 코드는 FAE-32(128개)입니다(`03` §3, `04` C7).
5. **LARYBench 비교 조건이 다릅니다.** 영상 기반 모델만 중간 프레임을 받았고, DreamX 학습 데이터의 37.4%인 AgiBot이 평가셋 AgiBotWorld-Beta와 같은 출처입니다(학습·평가 분리 여부 미보고).
6. **정책 실험의 표본과 기준선이 작습니다.** 과제 하나, 설정당 10회, 체크포인트 1개입니다. 기준선은 Mimic-Video 하나이고 사전학습 출발점이 다릅니다. 차이는 push에서만 나옵니다.
7. **재현 정보가 부족합니다.** 코드·가중치가 없고, 사전학습 연산량·배치·seed, 행동 레이블 클립 비율, 정책에 쓴 사람 영상 수가 빠져 있습니다(`04` §2).
8. **사소한 표기 문제:** VideoWorld2가 참고문헌에 같은 논문으로 두 번(Ren et al. 2026a, 2026b) 들어가 있습니다. LARY-Bench/LARYBench, MolmoAct2-BimanualYAM/MolmoAct-BimanualYAM, reach/approach처럼 같은 대상을 다르게 부릅니다.

03. 분석: 정렬 toy, LARYBench 읽기, 압축 산술, 실험 통계

이 문서의 계산은 `analysis/alignment_toy.py` (numpy, 단일 스레드, 약 20초)와 `analysis/table_check.py` (1초 미만)가 만듭니다. 결과는 `alignment_toy_results.json`, `table_check_results.json`에 있습니다.

1. 재구성만으로 사람·로봇 latent action이 정렬되는가 (toy)

1.1 질문

DreamX가 내세우는 전이(Eq. 5)는 사람 영상에서 뽑은 Q_src를 로봇의 행동 branch로 decode합니다. 이것이 되려면 **같은 행동이 사람과 로봇에서 같은 latent 좌표**로 가야 합니다. 그런데 학습 손실(Eq. 4)에는 이를 직접 요구하는 항이 없습니다.

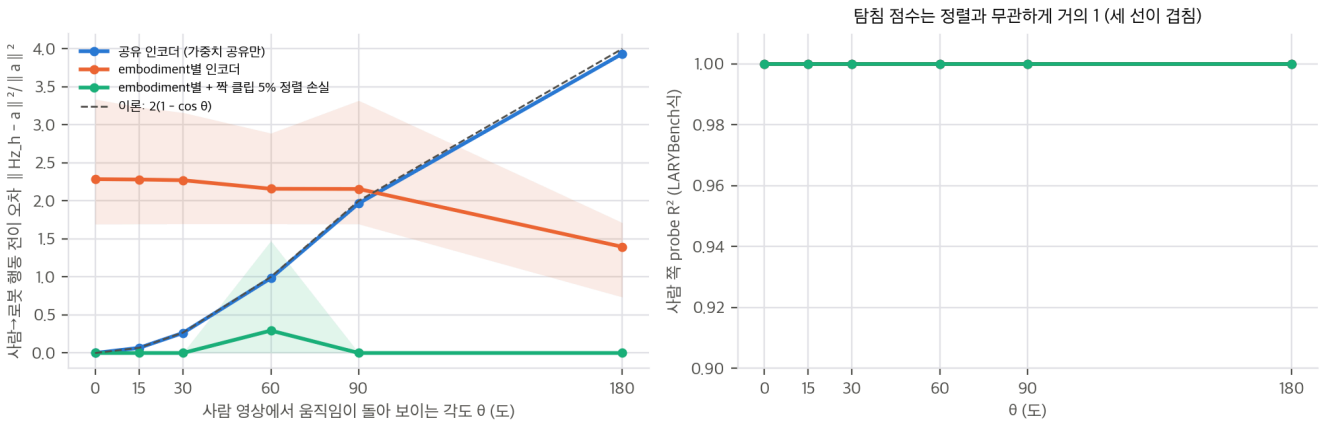
- 재구성 손실은 embodiment마다 latent를 가역 변환해도 값이 같습니다. decoder가 첫 프레임 V_0로 embodiment를 알 수 있으므로, 사람용 좌표계와 로봇용 좌표계를 따로 써도 재구성은 똑같이 잘 됩니다.
- 행동 보조 손실은 로봇 클립에만 있습니다(사람은 $\lambda = 0$). 그래서 로봇 latent만 행동 좌표에 묶입니다.

남는 정렬 압력은 두 가지입니다. (a) encoder 가중치 공유, (b) 같은 행동이 이미지에서 비슷하게 움직여 보이는 것. toy는 이 둘이 각각 얼마나 일을 하는지 봅니다.

1.2 모형

- 행동 $a \in \mathbb{R}^2$, 영상 공간의 움직임 $\Delta o_e = R_e\, a$ + 잡음(8차원).
- 로봇 렌더러 R_r은 직교 열 2개입니다. 사람 렌더러는 $\mathbf{R_h = R_r\, Q(\theta) + \eta\, R_\perp}$ 를 열마다 정규화한 것입니다. θ는 시점 차이 때문에 이미지 평면에서 움직임이 돌아 보이는 각도, $\eta = 0.3$은 손이 그리퍼와 다르게 보이는 정도입니다.
- encoder $z = E_e\, \Delta o$, decoder $\Delta o \approx D_e\, z$(embodiment별), 로봇 클립에만 행동 head 손실 $\| H\, z - a \|^2$($\lambda = 1$, H 공유)를 둡니다. DreamX의 Eq. 3-4를 선형으로 줄인 구조입니다.
- 조건 세 가지:
  - **S:** 공유 encoder. 가중치 공유가 유일한 정렬 압력입니다.
  - **I:** embodiment별 encoder. 강한 비선형 encoder가 입력만 보고 embodiment를 알아보는 경우를 근사합니다.
  - **P:** embodiment별 encoder에, 짝지은 사람·로봇 클립 5%로 정렬 손실 $\| z_h - z_r \|^2$(가중치 5)를 더합니다. DreamX가 쓰지 않는 신호입니다.
- 지표: (1) **탐침 R^2**: embodiment마다 $z \to a$ 선형 회귀를 따로 학습합니다. LARYBench 방식입니다. (2) **전이 오차** $\| H\, z_h - a \|^2 / \| a \|^2$: 사람 latent를 로봇 행동 head로 풀었을 때의 오차입니다. DreamX의 행동 전이(Eq. 5)에 해당합니다.
- Adam(lr 0.02, weight decay 1e-3) 2,500 step, embodiment당 1,000표본, seed 5개.

1.3 결과



선형 toy(행동 2차원, 영상 움직임 8차원), 시드 5개 평균과 최소-최대 범위. 로봇 클립에만 행동 보조 손실(DreamX 식 4). analysis/alignment_toy.py

왼쪽: 시점 회전 θ에 따른 사람→로봇 행동 전이 오차(seed 5개 평균, 음영은 최소-최대). 오른쪽: 사람 쪽 탐침 R^2(세 조건이 겹쳐 모두 1).

θ	0°	15°	30°	60°	90°	180°
S 공유 encoder	0.0003	0.067	0.264	0.984	1.968	3.935
이론 2(1 − cos θ)	0	0.068	0.268	1.0	2.0	4.0
I embodiment별 encoder	2.29	2.28	2.27	2.16	2.16	1.40
P + 짝 클립 5%	0.0001	0.0002	0.0001	0.295	0.0002	0.0002
탐침 R^2(세 조건 모두)	1.00	1.00	1.00	1.00	1.00	1.00

전이 오차 1은 "아무것도 예측하지 않는 것"(z_h = 0)과 같은 수준입니다. I의 seed별 값은 0.74-3.34 사이에서 흩어집니다. P의 60°는 seed 5개 중 1개가 국소 최소에 걸린 값(1.47)이고 나머지 4개는 0입니다.

1.4 읽기

1. **공유 encoder는 이미지 공간의 움직임이 일치할 때만 정렬됩니다.** S의 전이 오차는 이론값 2(1 − cos θ)를 거의 정확히 따릅니다. 행동 head를 로봇에 맞춰 H E R_r = I로 학습하면 사람 쪽은 H E R_h ≈ Q(θ)가 되고, 등방 행동에서 기대 오차가 ‖ Q(θ) − I ‖²_F / 2 = 2(1 − cos θ)이기 때문입니다. 사람 전용 방향 R_⊥을 써서 회전을 보정하는 해가 선형 encoder에도 존재하지만, 재구성 손실은 그 해를 고를 이유가 없고 weight decay는 오히려 막습니다.
2. **encoder가 embodiment를 알아볼 수 있으면 정렬은 임의적입니다.** I는 θ와 무관하게 오차가 1을 넘습니다. 재구성 손실이 embodiment별 좌표계를 허용한다는 것을 그대로 보여 줍니다. DreamX의 encoder는 2B 백본에 지시문까지 받으므로, 입력만으로 사람인지 로봇인지 쉽게 알 수 있습니다. 실제 모델은 S와 I 사이 어딘가에 있고, 어느 쪽에 가까운지는 사전학습 백본의 귀납 편향에 달려 있습니다.
3. **짝지은 클립이 조금만 있어도 정렬됩니다.** 5%로 θ와 무관하게 전이 오차가 0 근처로 갑니다. 정렬을 만드는 가장 싼 신호입니다.
4. **탐침 점수는 정렬을 재지 못합니다.** 세 조건, 모든 θ에서 사람 쪽 탐침 R^2는 1입니다. LARYBench처럼 탐침을 embodiment(벤치마크)마다 따로 학습하면, latent에 정보가 있는지는 재도 같은 좌표에 있는지는 재지 못합니다. Table 1의 높은 점수는 정렬의 근거가 아닙니다.

1.5 DreamX 실험에 대입

- Fig. 5(HumanAndRobot)와 Fig. 6(실물 행동 전이)의 사람 영상은 로봇과 **같은 위쪽 시점**에서 찍었습니다. toy로 말하면 θ ≈ 0인 경우입니다. 이 조건에서는 공유 encoder만으로 전이가 되는 것이 자연스럽고, DreamX의 결과와 맞습니다.

- 학습 데이터의 사람 영상(EgoDex, EgoVerse)은 1인칭입니다. 1인칭 손 움직임을 3인칭 로봇 카메라로 옮기는 경우는 θ가 크고, 실제로는 회전이 아니라 투영까지 바뀝니다. 이 경우의 정량 평가는 논문에 없습니다.
- 그래서 "짝 데이터 없이 정렬된다"는 결과는 **"시점이 맞으면 가중치 공유로 정렬된다"** 로 읽는 것이 안전합니다. 시점이 다른 전이에서는 소량의 짝 클립(P)이나 시점 정규화가 필요할 가능성이 큽니다.

1.6 toy의 한계

선형 모형이고, 시점 변화를 이미지 평면 회전으로만 근사했습니다. 실제 영상 diffusion 백본의 사전학습 표현은 시점에 어느 정도 불변인 특징(예: 물체 중심 좌표)을 가질 수 있고, 그렇다면 S보다 나은 정렬이 나올 수 있습니다. toy가 보여 주는 것은 "손실이 정렬을 요구하지 않는다"까지입니다. 실제 정렬 정도는 시점이 다른 짝 데이터(예: 1인칭·3인칭 동시 촬영)로 직접 재야 합니다.

2. LARYBench가 재는 것과 재지 못하는 것 (Table 1, 부록 B.4)

2.1 이득의 크기

기준선 중 열별 최고값 대비 DreamX의 개선입니다(`table_check.py` C1).

열	최고 기준선	DreamX	상대 개선	FAE-32 상대 개선
CALVIN 회귀	0.43	0.15	−65.1%	−46.5%
VLABench 회귀	0.09	0.05	−44.4%	−22.2%
RoboCOIN 회귀	0.51	0.26	−49.0%	−39.2%
AgiBotWorld-Beta 회귀	0.52	0.22	−57.7%	−50.0%
Atomic robot 분류	58.51	68.03	+9.52%p	+7.39%p
Composite human 분류	65.47	72.72	+7.25%p	+4.02%p
Composite robot 분류	62.76	68.26	+5.50%p	+3.30%p

회귀에서는 이득이 크고 모든 열에서 일관됩니다. 분류 이득은 상대적으로 작습니다.

2.2 같은 조건 비교가 아닌 이유

1. **입력 프레임 수:** 회귀 벤치마크는 원래 표본당 프레임 2장입니다. 영상 기반 모델에는 저자들이 원본에서 중간 프레임을 복원해 넣었습니다(부록 B.4). 위 그룹(프레임 2장)과 아래 그룹(영상)의 차이 일부는 입력 정보량 차이입니다.
2. **특징 크기:** DreamX 특징은 latent frame마다 2,048차원 토큰 4개입니다. 탐침은 토큰마다 768차원으로 사영한 뒤 attentive pooling을 하므로, 넓은 특징이 유리할 수 있습니다. 그래서 **토큰당 32차원인 FAE-32가 더 공정한 비교 대상**입니다. FAE-32도 모든 기준선을 이기므로, 결론의 방향은 유지됩니다.
3. **데이터 출처 겹침:** DreamX 사전학습의 37.4%가 AgiBot이고(Fig. 9), 회귀 평가셋 AgiBotWorld-Beta도 같은 출처입니다. 다만 사전학습에 없는 CALVIN에서 상대 개선이 가장 크므로(−65.1%), 겹침만으로 설명되지는 않습니다.
4. **텍스트:** encoder가 지시문 ℓ을 받는데(Eq. 2), 탐침용 특징을 뽑을 때 무엇을 넣었는지 적혀 있지 않습니다(`04` C9).
5. **정렬 미측정:** §1.4대로, 벤치마크마다 따로 학습한 탐침은 embodiment 간 정렬을 재지 못합니다.

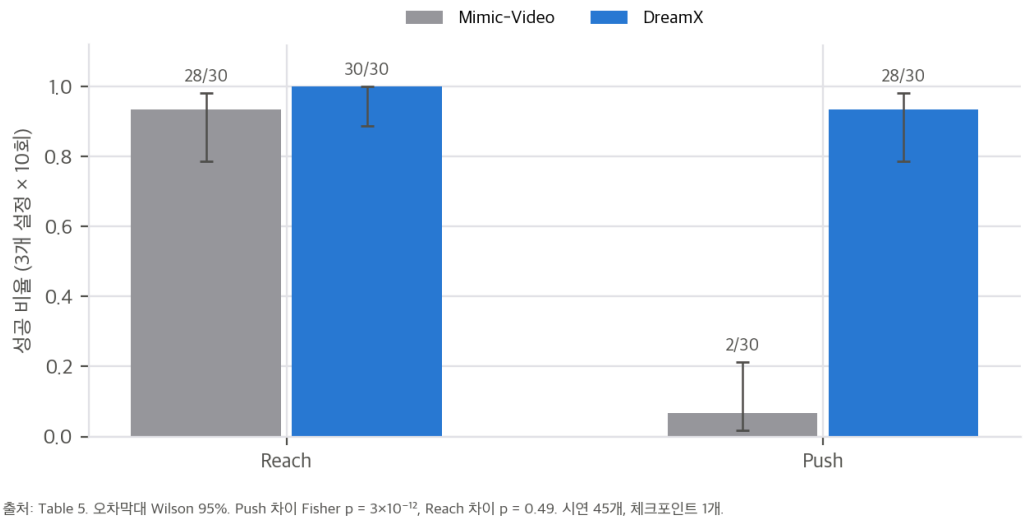
3. "300× 압축"의 산술 (`table_check.py` C7)

Cosmos-Predict2.5의 VAE를 공간 8배·시간 4배·16채널, patch 2로 두었습니다. 이 사양은 논문에 없어서 사전 지식으로 넣은 가정입니다. 이 가정으로 논문의 "latent frame당 1,200토큰"과 "29프레임 → latent frame 8개"가 그대로 나옵니다.

표현	프레임당 개수	프레임당 실수 개수	VAE latent 대비
백본 토큰 (30 × 40 패치, 폭 2,048)	1,200	2,457,600	—
VAE latent (60 × 80 × 16)	—	76,800	1×
DreamX latent action (토큰 4 × 2,048)	4	8,192	9.4× 작음
FAE-32 압축 코드 (토큰 4 × 32)	4	128	600× 작음

- "1,200 → 4토큰, 300×"(초록, §1)는 **토큰 개수의 비**입니다. 실수 개수로는 백본 토큰 대비 300배이지만, 영상 자체(VAE latent) 대비로는 9.4배에 그칩니다.
- 논문도 §3.4에서 "토큰 수는 적지만 각 토큰이 백본 폭을 그대로 가진다"고 인정하고 FAE로 줄입니다. 그러니 초록과 프로젝트 페이지의 "300× fewer tokens"는 정확하지만, "compact"한 행동 코드라고 부를 수 있는 것은 FAE-32입니다.
- 정책 지연 19.1배 단축(Table 3)은 이 압축 덕분이라기보다, 미래 영상(수만 개 실수)을 생성하지 않고 4 × 32 코드만 생성하는 데서 옵니다. 지연 비는 28,226 / 1,474 = 19.15입니다.

4. 실물·CALVIN 결과의 통계 (`table_check.py` C3-C5)



출처: Table 5. 오차막대 Wilson 95%. Push 차이 Fisher $p = 3\times10^{-12}$, Reach 차이 $p = 0.49$. 시연 45개, 체크포인트 1개.

Table 5를 세 설정 합산으로 다시 그림. 오차막대는 Wilson 95% 구간.

비교	성공 수	Wilson 95%	Fisher p
push 합산: DreamX vs Mimic-Video	28/30 vs 2/30	[0.79, 0.98] vs [0.02, 0.21]	3.2×10^{-12}
reach 합산	30/30 vs 28/30	—	0.49
위 서랍 push	8/10 vs 2/10	—	0.023
사람 영상만 vs 로봇 시연 (위 서랍 push, Table 6 vs 5)	10/10 vs 8/10	—	0.47

- **DreamX가 push에서 앞선다는 결과는 표본이 작아도 확실합니다.** 다만 Mimic-Video가 30회 중 28회 서랍에 닿고도 2회만 닫는 것은 "덜 학습된 기준선"일 가능성도 시사합니다. 두 방법의 사전학습 출발점(MolmoAct 영상 백본 대 DreamX 생성기)과 사전학습 데이터 규모가 다르고, 기준선 튜닝 절차는 적혀 있지 않습니다.
- **"사람 영상만으로도 100%"** 는 10회 표본에서 로봇 시연(8/10)과 구분되지 않습니다. 정확히 말하면 "사람 영상으로 바꿔도 떨어지지 않았다"입니다.
- **CALVIN(Table 7):** Avg. Len.은 ≥k 열의 합(÷100)으로 정확히 재현됩니다(2.378, 2.643). 평가 시퀀스 수가 없어서, 관례인 1,000개로 가정하면 ≥1 차이는 $p = 0.17$, ≥5 차이는 $p = 0.0057$입니다. 긴 과제열로 갈수록 차이가 벌어지는 패턴이라 효과

는 있어 보이지만, 같은 정책 안에서 latent 특징 attention을 막는 ablation이라 "latent action 예측이 행동 학습을 돕는다"까지만 말할 수 있습니다. 절대 수준 2.64도 CALVIN의 최근 정책들보다 낮습니다.

5. Table 8 읽기: latent action은 움직임을 얼마나 결정하는가 (`table_check.py` C8)

조건	움직임 비율	flow 편차 (px/프레임)	RGB L1	정지 영역 RGB L1	정지/전체
α = 0.1	1.000	0.141	0.0054	0.0030	0.56
α = 0.3	0.995	0.211	0.0077	0.0034	0.44
α = 1.0	0.946	0.481	0.0215	0.0077	0.36
query = 0	0.456	0.866	0.0758	0.0227	0.30

1. 교란이 커질수록 flow 편차가 단조 증가하고, 정지 영역의 변화는 전체보다 작습니다. "latent action이 외관보다 움직임을 조종한다"는 해석과 맞습니다.
2. 그러나 **채널 표준편차만큼의 잡음(α = 1)에도 움직임 크기는 94.6% 유지됩니다.** decoder가 latent의 세부 값에 둔감하다는 뜻입니다. 행동 전이처럼 latent의 세부가 궤적을 결정해야 하는 용도에서는 이 둔감함이 정밀도의 상한이 될 수 있습니다.
3. **query를 0으로 바꿔도 움직임이 45.6% 남습니다.** 첫 프레임과 생성 prior만으로도 "그럴듯한 움직임"이 상당히 나옵니다. Table 2의 영상 전이 점수에도 이 몫이 섞여 있을 수 있으므로, query = 0 기준선의 FVD·PSNR이 함께 있어야 합니다(`04` C2).
4. 표본은 **골라 낸 고움직임 클립 8개 × seed 2개**입니다. 분포를 대표하지 않습니다.

추가 검증 (2026-11-11)

스크립트: `analysis/extra_alignment_nonlinear.py` → `analysis/extra_alignment_nonlinear_results.json` (PyTorch CPU, 4 스레드, 실행 57초). 선형 정렬 toy(§1)를 비선형으로 바꾼 실험입니다.

추가 검증 1: 비선형 encoder·decoder에서도 공유 encoder의 정렬이 시점 각도에 따라 깨지는가

목적. §1의 선형 결론 세 가지(공유 encoder는 θ ≈ 0에서만 정렬, embodiment별 encoder는 임의적, 짝 클립 5%면 정렬)가 비선형 encoder와 행동 손실에서도 유지되는지 봅니다. 탐침 R^2가 정렬을 재지 못한다는 결론도 함께 봅니다.

설정(가정).

- 행동 $a \in R^2$, 관측 $\Delta o_e = \tanh(R_e\, \varphi(a)\, W + \text{잡음}) \in R^8$, $\varphi(a) = [a_1, a_2, a_1a_2, \sin a_1]$. 로봇 R_r은 직교 열 2개, 사람 $R_h = R_r\, Q(\theta) + 0.3\, R_\perp$(열 정규화).
- encoder MLP 8→64→2, embodiment별 decoder MLP 2→64→8(재구성). 로봇 클립에만 행동 head 손실 $\| Hz - a \|^2$(H 선형, 공유).
- 조건: S 공유 encoder, I embodiment별 encoder, P embodiment별 encoder + 짝 클립 5%에 가중치 5의 $\| z_h - z_r \|^2$.
- 지표: 탐침 R^2(사람 latent → 행동, 선형), 전이 오차 $\| Hz_h - a_h \|^2 / \| a_h \|^2$. embodiment당 1000표본, Adam 2000 step, $\theta \in \{0, 30, 60, 90\}°$, 시드 3개.
- 첫 프레임 조건 decoder는 두지 않았습니다(embodiment별 decoder로 대신). CNN 대신 MLP.
- 장치: CPU. 패키지: torch 2.15.0, numpy 1.26.4, Python 3.12.7.

결과 (시드 3개 평균).

θ	0°	30°	60°	90°
S 전이 오차	0.21	1.09	2.79	2.84
I 전이 오차	1.64	1.33	1.40	1.23
P 전이 오차	0.22	0.28	0.27	0.25
S 사람 탐침 R^2	0.80	0.59	0.40	0.41
I 사람 탐침 R^2	0.65	0.61	0.60	0.61
P 사람 탐침 R^2	0.78	0.73	0.74	0.75

(전이 오차 1은 대략 "행동을 예측하지 않는" 수준입니다. 시드별 값은 결과 JSON에 있습니다.)

결론.

- 공유 encoder의 정렬은 θ에 따라 깨집니다. 전이 오차가 0.21(θ = 0)에서 2.8(θ ≥ 60°)로 올라갑니다. 선형 toy의 결론과 같은 방향입니다.
- 짝 클립 5%를 넣은 P는 θ와 무관하게 0.25-0.28로 정렬됩니다. 선형 toy와 같은 결과입니다.
- embodiment별 encoder(I)는 θ와 무관하게 1.2-1.6으로 정렬되지 않습니다. 선형 toy와 같은 방향입니다.
- **탐침 점수에 대한 결론은 이 비선형 설정에서 유지되지 않습니다.** 선형 toy에서는 탐침 R^2가 세 조건과 모든 θ에서 1이었지만, 비선형에서는 S의 탐침 R^2가 θ에 따라 0.80에서 0.41로 떨어지며 전이 오차와 같이 움직입니다. 반면 P는 탐침 R^2가 0.75 근처이면서 전이 오차는 가장 작습니다. 따라서 탐침 점수 하나로 정렬을 판단할 수 없다는 주장은 맞지만, 탐침이 정렬과 무관하다는 강한 주장은 이 실험에서 지지되지 않습니다.

한계. 영상 대신 8차원 움직임 벡터, CNN 대신 MLP, 첫 프레임 조건 없음, 시드 3개입니다. 전이 오차의 S와 I 값은 시드 간 편차가 큽니다(S, θ = 90°: 2.46-3.33). 논문의 DreamX 백본과 직접 비교할 수 없습니다.

04. 주장 검증: 수치 재계산, 공개 자료 대조, 구현 가능성

- **대상:** arXiv:2611.05377v1 본문과 부록 A-D, 프로젝트 페이지(HTML, 2026-11-09 저장).
- **공식 코드:** 없습니다. 프로젝트 페이지는 "Code soon"이고, `cvlab-kaist/DreamX` 저장소(@ `056bd46`, push 2026-11-06, 라이선스 없음)에는 README("Code-Coming Soon")와 티저 그림만 있습니다. 그래서 코드 대조 대신 표 재계산, 프로젝트 페이지 대조, 공개 백본 사양 확인, numpy toy로 검증했습니다.
- **대조한 외부 자료:**
  - 백본 사양: `nvidia-cosmos/cosmos-predict2.5` `docs/setup.md` (main @ `a2c298b`), HF `nvidia/Cosmos-Predict2.5-2B` 메타데이터(파일 크기, 라이선스 태그만, 가중치는 받지 않음)
  - 관련 LAM 저장소 메타데이터(`02` §4)
- **스크립트:** `analysis/table_check.py`(표 재계산, Fisher·Wilson), `analysis/alignment_toy.py`(정렬 toy). 둘 다 CPU 단일 스레드, numpy·scipy만 씁니다.

1. 수치 재계산과 주장 판정

#	주장·수치 (출처)	재계산·확인	판정
C1	Table 1 평균과 "모든 벤치마크에서 최고"(§4.1)	13개 방법의 분류 평균은 모두 일치. 회귀 평균은 DiLA만 0.3875로 표의 0.38과 경계(원값이 반올림 전 더 작아야 가능). DreamX는 8열 모두 1위, 회귀 상대 개선 44-65%, 분류 +5.5-9.5%p. FAE-32도 8열 모두 기준선보다 나음	✅ 수치 일치(경계 1건). 비교 조건은 C9· `03` §2
C2	Table 2 "모든 지표에서 앞섬"(§4.2)	FVD 2.96배 낮음, LPIPS −40.1%, PSNR +3.57 dB, SSIM +0.036. 1,008클립 × 19프레임. seed·신뢰구간 없음. 첫 프레임 복사, query = 0 같은 자명한 기준선 없음. query = 0에서도 움직임이 45.6% 남으므로(Table 8) 이 기준선이 필요	✅ 순위 / ⚠️ 움직임 전이 몫은 분리 안 됨
C3	Table 3 SR 96.7 vs 50.0, "19.1배 빠름"(§4.4)	Table 5에서 설정별 SR 90/100/100, 50/50/50을 정확히 재현. 지연 비 19.15. push 28/30 vs 2/30($p = 3.2 \times 10^{-12}$), reach 30/30 vs 28/30($p = 0.49$)	✅ 일치. 차이는 전부 push
C4	"사람 영상만으로 학습해도 100%"(부록 C.1, Table 6)	위 서랍 push 10/10 vs 로봇 시연 정책 8/10, $p = 0.47$. 사람 영상 개수 미보고	⚠️ "떨어지지 않았다"까지만
C5	Table 7 "latent 조건이 모든 길이에서 개선"(부록 C.2)	Avg. Len. 2.378, 2.643 정확히 재현. 평가 시퀀스 수 미보고. 1,000개 가정 시 ≥1 $p = 0.17$, ≥5 $p = 0.0057$	✅ 방향 / ⚠️ 표본 수 불명, 절대 수준 낮음
C6	"2.19k시간, 사람 469시간"(부록 A.1, Fig. 9)	합계 2,188.0 h, 사람 469.1 h(21.4%). AgiBot 818.2 h(37.4%)가 LARYBench AgiBotWorld-Beta와 같은 출처, 분리 여부 미보고	✅ 합계 / ⚠️ 평가 셋 출처 겹침
C7	"latent frame당 300× 압축(1,200 → 4토큰)"(초록, §1)	VAE 8×·4×·16채널, patch 2 가정으로 1,200토큰과 latent frame 2-8개가 재현됨. 토큰당 2,048차원이라 프레임당 8,192개 실수, VAE latent(76,800) 대비 9.4×. FAE-32는 128개로 600×	✅ 토큰 수로는 맞음 / ⚠️ 정보량 압축으로 읽으면 과장
C8	"latent action이 외관을 유지하며 움직임을 조종"(§4.6, Table 8)	flow 편차 단조 증가, 정지/전체 RGB 변화 0.56 → 0.30. α = 1에서도 움직임 94.6% 유지, query = 0에서도 45.6% 남음. 골라 낸 클립 8개 × seed 2	✅ 방향 / ⚠️ decoder가 latent 세부에 둔감, prior 몫 큼

#	주장·수치 (출처)	재계산·확인	판정
C9	"짝지은 데이터 없이 전이, 범용 latent action" (§3.3, §4.2-4.3)	(a) encoder는 지시문 ℓ을 받고 decoder는 받지 않음(Eq. 2, §3.2). 전이와 LARYBench 추출에서 무엇을 ℓ로 넣었는지 미보고, 빈 지시 ablation 없음. (b) Fig. 5·6의 사람 영상은 로봇과 같은 위쪽 시점. (c) toy에서 재구성 손실은 정렬을 요구하지 않고, 공유 encoder는 이미지 움직임이 일치할 때만 정렬됨(`03` §1)	⚠️ 시점 일치 조건에서만 근거 있음, 텍스트 경로 미통제
C10	"행동 레이블 없이 영상만으로 행동 의미를 잡는다"(§4.5, Table 4)	0.24/65.3 vs 0.23/64.8, seed 1개, 일부 데이터. 탐침 점수만 재고 전이(정렬)는 재지 않음. toy에서 탐침 R^2는 정렬 여부와 무관하게 1	⚠️ 정렬에 대한 행동 손실의 역할은 미검증
C11	"로봇 간 공통 방향 구조"(부록 C.3, Fig. 12)	로봇별 평균을 뺀 방향별 중심점 60개의 t-SNE. 개별 클립이 아닌 집계이고 사람 클립 없음. 로봇별 평균 제거가 embodiment 오프셋을 직접 지움	⚠️ 약한 근거
C12	실물 정책 비교의 공정성(부록 B.3)	Mimic-Video는 MolmoAct 영상 백본, DreamX는 사전학습 생성기(사전학습 데이터 약 1/3, H200 2장 100k step)에서 출발. 둘 다 30k step. Mimic-Video는 reach 28/30인데 push 2/30. 기준선 튜닝 절차, 학습 seed 미보고	⚠️ 출발점이 달라 "latent 예측 대 영상 예측" 비교로 단정 못 함
C13	프로젝트 페이지 수치	"300× fewer tokens", MSE 0.17 vs 0.38, FVD 57.4 vs 170.2, 2.19k h, 막대그래프 9개 방법 값이 Table 1·2와 일치	✅ 일치

2. 재현에 필요한 정보 체크리스트

항목	논문에 있음	없음
모델 구성	백본(Cosmos-Predict2.5 2B), query 4개/latent frame, LoRA rank 16(encoder·decoder 각각), 마스크 규칙(Fig. 2b), FAE 구조(self-attn 1블록, 6층 decoder, 32차원)	행동 branch의 구조(행동 토큰 수, chunk 길이, 사영층), Action DiT 크기, query 초기화
학습 하이퍼파라미터	step 수(78k, 50k, 100k, 30k), 학습률, warm-up, $\beta = 0.01$, 정책 생성기 배치 60·H200 2장	DreamX 사전학습 배치, GPU 수·시간, seed, flow timestep 분포, 행동 손실의 정확한 정규화
데이터	6개 데이터셋과 시간(Fig. 9), 해상도, 클립 길이	데이터셋별 표집 비율, 행동 레이블이 있는 클립 비율과 행동 공간 통일 방식(데이터셋마다 다름), 지시문 출처(사람 영상), 평가셋과의 분리
평가	LARYBench 탐침 구조, H&R 클립 수·지표·horizon, 실물 시행 수	탐침 학습 설정과 seed, LARYBench 추출 때 지시문, CALVIN 평가 시퀀스 수와 학습 split, 실물 사람 영상 개수
공개물	백본(공개)	코드, DreamX 가중치, FAE, 정책 체크포인트, 실물 데이터

3. 대조 메모

- 프로젝트 페이지의 표와 막대그래프 값은 Table 1과 칸마다 같습니다(C13).
- 백본 공개 상태: `nvidia/Cosmos-Predict2.5-2B`는 NVIDIA Open Model License(HF 태그 `license:other`, 자동 승인 gated)입니다. bf16 EMA 체크포인트가 4.12 GB, tokenizer(VAE)가 0.51 GB입니다. 같은 저장소에 `robot/policy/libero`, `robot/policy/robocasa` 체크포인트가 있습니다. 영상 백본을 정책으로 미세조정하는 Cosmos Policy(Kim et al. 2026b)의 공개 사례로 보입니다. DreamX §3.4는 "같은 영상 백본으로 미래 영상 대신 latent action을 예측한다"는 문장에 Kim et al. 2026a를 인용하는데, 참고문헌의 2026a는 "Planning in 8 tokens"(압축

tokenizer)이고 영상 백본을 정책으로 바꾸는 논문은 2026b(Cosmos Policy)입니다. 인용 표기가 바뀌었을 가능성이 있습니다(`02` §5).

- 공식 Cosmos 실행 환경은 NVIDIA Ampere 이상 GPU, CUDA 12.8.1, Linux x86-64를 요구합니다(`docs/setup.md`).

4. 구현할 수 있는가

4.1 공식 재현

불가능합니다. 코드와 가중치가 없습니다. 데이터 6종은 모두 공개 데이터셋이지만(윤리 절), 합치면 2.19k시간이고 DROID·AgiBot만 해도 수 TB 규모입니다. DreamX 사전학습의 GPU 수와 시간은 적혀 있지 않고, 정책 생성기만 H200 2장으로 100k step입니다. 실물 정책 결과는 양팔 YAM과 비공개 시연 45개가 있어야 합니다.

4.2 이 맥(Apple M4 Pro, 통합 메모리 약 24 GiB, MPS, CUDA 없음)

작업	판단	이유
표 재계산, 통계 검정, 정렬 toy	✅ 완료	`table_check.py` 1초 미만, `alignment_toy.py` 약 20초
공식 DreamX 추론	❌	코드·가중치 없음
Cosmos-Predict2.5 2B 단독 추론	⚠️ 조건부, 미검증	가중치 bf16 4.12 GB + VAE 0.51 GB는 메모리에 들어감. 480 × 640, latent frame 8개면 토큰 약 9,600개라 SDPA 추론 활성값도 들어갈 것으로 추정. 다만 공식 스택이 CUDA·Linux 전제이고, 텍스트 encoder(Cosmos-Reason1 7B로 알려짐, 사전 지식)는 bf16로 약 15 GB라 함께 올리기 어려움. 지시문 임베딩을 미리 계산하거나 빈 문자열 임베딩(`cr1_empty_string_text_embeddings.pt`)으로 대체해야 함. torch 설치와 약 4.6 GB 다운로드(라이선스 동의) 필요
DreamX 방식 재구현 + 학습(2B, LoRA)	❌ 사실상 불가	시퀀스 약 9,600토큰의 2B 모델을 LoRA로 역전파하면 MPS에서 step당 수십 초 이상으로 추정. 78k step과 2.19k시간 데이터는 비현실적
축소 재구현(작은 영상 모델 + query + 재구성, 합성 데이터)	✅ 가능	torch(MPS) 설치 후 수 시간 규모. 정렬 질문(`03` §1)을 비선형 모델로 확인하는 데 충분

4.3 아이디어 재구현 (CUDA GPU가 있는 경우)

구조 자체는 간단합니다. 공개 백본에 (1) latent frame당 query 4개, (2) Fig. 2b 마스크, (3) encoder·decoder 각각 LoRA, (4) 텍스트 cross-attention 자리에 query 조건, (5) 로봇 클립용 행동 branch를 붙이면 됩니다. 어려운 점은 세 가지입니다.

1. **decoder의 조건 경로:** 원래 텍스트 cross-attention이 있던 자리에 query를 넣는지, 영상 토큰 열에 이어 붙이는지(논문은 "temporal sequence를 따라 이어 붙인다", §3.2)에 따라 구현이 달라집니다. 행동 branch의 구조는 적혀 있지 않습니다.
2. **행동 공간 통일:** 로봇 데이터 4종은 행동 공간이 서로 다릅니다. 어떻게 하나의 행동 branch로 학습했는지 설명이 없습니다.
3. **연산량:** 2B 백본, 480 × 640, 최대 29프레임, 78k step이면 H100급 여러 장이 필요할 것으로 추정됩니다. 줄인 해상도와 일부 데이터(Table 4처럼 MolmoAct2·EgoVerse·EgoDex 일부)로 시작하는 것이 현실적입니다.

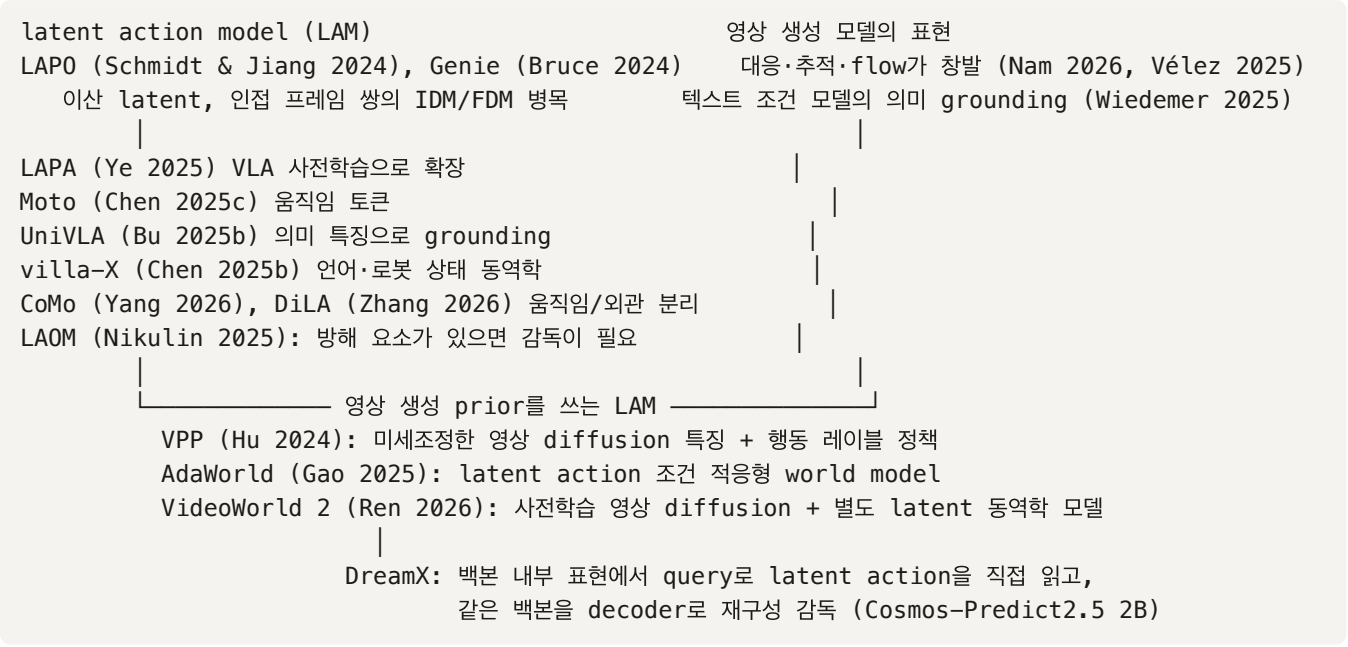
4.4 판단

- **공식 재현: 불가능**(코드·가중치 미공개, 사전학습 연산량 미보고).
- **이 맥: 분석·toy까지 완료.** 공개 백본 단독 추론은 설치·다운로드 승인 후 조건부(미검증)이고, DreamX 학습은 불가능합니다.

- **재구현 난이도: 중간.** 공개 백본에 query + LoRA를 붙이는 구조라 코드량은 적지만, 행동 branch와 행동 공간 통일 방식이 빠져 있고 CUDA GPU 여러 장이 필요합니다. 가장 먼저 확인할 가치가 있는 것은 성능보다 **시점이 다른 사람→로봇 전이에서 정렬이 유지되는지**입니다(`03` §1).

02. 선행연구와 위치

1. 계보



저자들의 위치 설정(§2)은 이렇습니다. 기존 LAM은 프레임 한 쌍 같은 국소 전이에서 행동 공간을 처음부터 학습하므로, 움직임이 한 step 너머로 어떻게 이어지는지 보지 못하고 외관과 행동이 얽힌다고 봅니다. 영상 prior를 쓰는 LAM(AdaWorld, VideoWorld 2)도 영상 모델은 픽셀이나 rollout을 공급할 뿐이고, 행동 표현은 처음부터 학습한 외부 모듈이 만든다고 봅니다. DreamX는 백본 내부 표현을 직접 읽고, embodiment별 행동 encoder를 쓰지 않는다는 점을 차이로 듭니다.

이 위치 설정에서 빠진 것. 저자들이 인용한 LAOM(Nikulin et al. 2025)의 결론은 "방해 요소가 있으면 latent action 학습에 감독이 필요하다"입니다. DreamX는 이 문제를 "생성 prior가 외관을 맡는다"로 우회한다고 주장하지만, 사람 쪽에는 감독이 전혀 없고 로봇 쪽 행동 손실만 있습니다. 감독 없이 사람·로봇 latent가 같은 좌표에 놓이는 이유는 설명하지 않습니다(`03` §1).

2. 구성 요소별 출처

구성 요소	출처	DreamX에서 바꾼 점
영상 diffusion 백본	Cosmos-Predict2.5 2B (Ali et al. 2025)	encoder와 decoder 양쪽으로 재사용, 각각 rank-16 LoRA, 백본 고정
query readout	learnable query로 특징을 모으는 Perceiver·Q-Former 계열 방식	영상 토큰 열에 붙여 백본 시공간 attention을 함께 통과, latent frame당 4개, query → 영상은 프레임 causal
latent action + 미래 재구성	LAPO, Genie, LAPA의 IDM/FDM 병목	decoder를 사전학습 영상 diffusion으로 바꾸고, 텍스트 조건 자리를 latent action이 대신함
행동 보조 branch	영상·행동 결합 생성(WAM 계열)	레이블 있는 클립만 $\lambda > 0$, 영상과 행동 스트림이 서로를 보지 않음
압축 코드(FAE)	Gao et al. 2026 "One layer is enough"의 feature autoencoder	토큰 배치를 유지하며 2,048 → 32차원, $\beta = 0.01$
정책 생성기	영상 백본을 정책으로 미세조정하는 방식(Cosmos Policy, Kim et al. 2026b)	미래 영상 대신 압축 코드 Z를 생성, 별도 Action DiT가 단방향으로 읽음

구성 요소	출처	DreamX에서 바꾼 점
기준선 정책	Mimic-Video (Pai et al. 2025)	MolmoAct로 사전학습한 영상 백본에서 출발
평가	LARYBench (Nie et al. 2026), HumanAndRobot (Human2Robot, Xie et al. 2026), CALVIN	LARYBench 영상 모델에는 중간 프레임 복원
데이터	MolmoAct2-BimanualYAM, Bridge, DROID, AgiBot, EgoDex, EgoVerse	2.19k시간, 사람 21.4%

3. 이 저장소의 다른 논문과의 관계

논문	관계
ThinkWorld (`thinkworld/`)	**같은 그룹의 짝 논문입니다.** 저자 4명(Chaehyun Kim, Jisoo Kim, Seungryong Kim, Fu-En Yang)이 겹치고, 둘 다 양팔 YAM과 MolmoAct2-BimanualYAM, 사람 영상 EgoDex·EgoVerse를 씁니다. 사람·로봇 공통 행동 인터페이스를 푸는 방식이 정반대입니다. ThinkWorld의 Chain-of-Dynamics는 **손목 자세 주석에서 만든 명시적 6D/12D 움직임 토큰**이라 정렬이 구성상 보장되지만 주석이 필요합니다(ThinkWorld 분석에서 "레이블 없는 사람 영상"이라는 서술이 손 자세 주석을 쓴다는 점을 지적했습니다). DreamX는 주석 없이 latent를 학습하지만 정렬이 보장되지 않습니다(`03` §1). 미래 예측 대상도 다릅니다. ThinkWorld는 1초 뒤 DINOv3 특징을, DreamX는 영상 diffusion으로 미래 프레임 자체를 재구성합니다. toy 결과로 보면 두 접근을 합치는 것이 자연스럽습니다. 사람 클립 일부에만 손목 자세 head를 붙여 latent를 고정하면 짝 클립(P 조건)과 같은 정렬 신호가 됩니다.
Hydra-0 (`hydra0/`)	Hydra-0는 행동을 **이미지 평면의 점 궤적(action flow)** 으로 명시적으로 표현해 Cosmos 계열 영상 world model의 조건으로 씁니다. DreamX는 같은 문제의 암묵적 버전입니다. 학습된 latent가 조건 역할을 합니다. 두 방법 모두 "이미지에서 보이는 움직임"이 공통 인터페이스라는 점은 같고, 그래서 둘 다 시점에 묶입니다. Hydra-0는 배포할 때 URDF·카메라 보정·Isaac Lab 투영이 있어야 하고, DreamX의 행동 전이는 로봇 초기 상태만 있으면 되지만 대응 관계가 보장되지 않습니다. 명시적 track은 시점이 달라도 3D 역투영으로 맞출 수 있다는 장점이 있습니다.
OpenWAM (`openwam/`)	OpenWAM은 영상 prior + 행동 생성(WAM)의 설계를 통제 실험으로 비교했고, 추론 때 미래 영상의 기여는 작고 이득은 학습 때 생긴다는 결과를 냈습니다. DreamX 정책은 이 관찰을 구조로 옮긴 사례입니다. 미래 영상 대신 4 × 32 압축 코드를 생성해 Mimic-Video보다 19.1배 빠릅니다. 다만 DreamX에는 "같은 백본, 예측 대상만 다름" 같은 통제 비교가 없어서(`04` C12), OpenWAM 같은 실험이 있어야 "latent 예측 대 영상 예측"을 판단할 수 있습니다. 사람 영상 비중은 OpenWAM 30%(비공개 1인칭), DreamX 21.4%(공개 EgoDex·EgoVerse)입니다.
AW-0 (`agentic_wam/`)	AW-0도 Cosmos-Predict2.5 2B 영상 전문가와 행동 전문가를 MoT로 묶어 함께 denoise합니다. DreamX 정책의 구조(영상 백본이 코드 Z 생성, Action DiT가 그 특징을 단방향으로 읽음)는 AW-0의 영상 → 행동 정보 흐름과 비슷합니다. AW-0 분석에서 "4단계 추론에서 행동이 읽는 미래 영상은 $\sigma \geq 0.625$의 흐린 상태"라고 지적했는데, DreamX는 영상 대신 작은 코드를 생성하는 쪽으로 이 문제를 피합니다. 사전학습 규모도 비슷합니다(AW-0 약 2,900시간, DreamX 2.19k시간).
PaceVLA (`pacevla/`)	PaceVLA는 텔레옵 시연의 느린 속도를 정책이 물려받는 문제를 다룹니다. DreamX의 latent action은 VAE의 시간 압축(latent frame 1개 = 영상 4프레임)에 묶여 있어, 같은 움직임이라도 사람(빠름)과 텔레옵 로봇(느림)에서 latent frame 수와 토큰 내용이 달라집니다. 행동 전이를 open-loop로 실행하면 사람의 속도가 그대로 로봇 명령으로 옮겨질 수 있는데, 논문은 속도·fps 정규화를 다루지 않습니다. 정책 1회 호출 1.47초라는 지연도 PaceVLA가 다루는 실행 속도 문제와 맞닿습니다.
GNR (`gnr/`)	GNR은 사람 손 시연을 로봇 손 궤적으로 **명시적으로** 리타겟팅합니다. 손 자세를 입력으로, MPC가 푼 궤적을 목표로 조건부 flow 생성기를 학습하고, 후보를 시뮬레이터로 거릅니다. DreamX의 행동 전이는 주석·기구학·물리 검사 없이 latent만으로 하고, 대상은 평행 그리퍼의 거친 움직임(글자 쓰기, 밀기)입니다. 물리적 타당성 검사가 없는 대신 입력 요구가 거의 없습니다. DreamX latent를 "움직임 의도"로, GNR식 시뮬레이터 거르기를 타당성 검사로 쓰는 조합을 생각할 수 있습니다.

DreamX가 위 논문들에 더하는 것

1. **주석 없는 공통 인터페이스의 실물 시연.** ThinkWorld(손목 자세), Hydra-0(점 궤적), GNR(손 자세 + MPC)은 모두 명시적 대응을 씁니다. DreamX는 주석 없이 사람 시연을 로봇 명령으로 옮긴 실물 사례를 보였습니다. 정량 평가가 없고 같은 시점 조건이라는 제약은 있습니다.

2. **영상 백본 내부를 읽는 latent action.** LAM 계열에서 처음으로 사전학습 영상 diffusion의 내부 표현에서 직접 latent action을 뽑았고, LARYBench에서 회귀 오차를 기존 최고 대비 절반 아래로 낮췄습니다.
3. **정책 지연.** 미래 영상 대신 압축 코드를 생성하는 WAM형 정책으로 지연을 28.2초(Mimic-Video)에서 1.5초로 줄였습니다.
4. **같은 약점.** 이 저장소의 다른 논문들처럼 같은 백본에서 한 변수만 바꾼 통제 비교, 시점이 다른 조건, 여러 seed가 없습니다.

4. 공개 저장소 (2026-11-09 확인, GitHub API 메타데이터)

저장소	역할	라이선스	★	마지막 push	확인한 커밋
cvlab-kaist/DreamX	공식 저장소	없음	5	2026-11-06	`056bd46` (README "Code-Coming Soon"과 티저 그림만)
nvidia-cosmos/cosmos-predict2.5	백본 코드	Apache-2.0 (가중치는 NVIDIA Open Model License)	1,394	2026-06-08	`a2c298b` (`docs/setup.md` 만 확인)
LatentActionPretraining/LAPA	LAPA, Table 1 기준선	MIT	587	2025-01-22	`46aca51`
OpenDriveLab/UniVLA	UniVLA, Table 1 기준선	Apache-2.0	1,145	2025-11-19	`0ab9e9d`
microsoft/villa-x	villa-X, Table 1 기준선	MIT	211	2026-09-16	`9b44899`
TencentARC/Moto	Moto, Table 1 기준선	NOASSERTION (GitHub가 판별 못 함)	181	2025-10-01	`63244f4`
Little-Podi/AdaWorld	AdaWorld, Table 1·2 기준선	Apache-2.0	264	2025-06-17	`fe3701e`
ByteDance-Seed/VideoWorld	VideoWorld 계열, Table 1·2 기준선	Apache-2.0	808	2026-02-25	VideoWorld 2 코드 포함 여부 미확인
senngadaisuki/disentangled-latent-action-world-models	DiLA, Table 1 최고 기준선	없음	21	2026-09-15	`adac869`
schmidtdominik/LAPO	LAPO (계보)	없음	149	2024-07-31	확인 안 함
dunnolab/laom	LAOM (Nikulin et al. 2025)	Apache-2.0	41	2025-07-08	`e133195`
mimic-video/mimic-video	Mimic-Video, 실물 기준선	Apache-2.0	317	2026-10-19	`a38dbfe`

- **Genie:** 공식 코드가 없습니다.
- **LARYBench:** 공식 저장소를 찾지 못했습니다(arXiv:2604.11689).
- 어떤 저장소도 clone하지 않았습니다. DreamX 코드가 없어서 04는 코드 대조가 아니라 주장 검증입니다.
- **HF:** `nvidia/Cosmos-Predict2.5-2B` (NVIDIA Open Model License, 자동 승인 gated, bf16 EMA 4.12 GB, tokenizer 0.51 GB). 메타데이터만 확인했습니다.

5. 참고문헌 메모 (확인 안 한 부분 포함)

- VideoWorld 2가 Ren et al. 2026a와 2026b로 두 번 들어가 있습니다(같은 arXiv:2602.10102). 본문에서 Table 1은 2026a, §2와 §4.2는 2026b를 인용합니다.
- §3.4의 "같은 영상 백본으로 미래 영상 대신 latent action을 예측(Kim et al. 2026a)"에서 2026a는 참고문헌상 "Planning in 8 tokens"(CVPR 2026, 압축 tokenizer)입니다. 영상 백본을 정책으로 미세조정하는 논문은 2026b(Cosmos Policy)라서, 인용이 바뀌었을 가능성이 있습니다.
- CoMo, VPP, Genie, LAPO의 세부 내용은 이번에 원문을 다시 읽지 않았습니다. 계보의 한 줄 설명은 DreamX §2의 서술을 따랐습니다.
- LARYBench 원 논문(arXiv:2604.11689)의 탐침 프로토콜과 Table 1 기준선 값이 원 논문 값인지 DreamX 저자들이 다시 돌린 값인지는 확인하지 않았습니다. 부록 B.4는 "모든 표현 모델을 고정하고 공통 탐침을 학습"한다고만 적었습니다.